# Native Delegation as a Runtime Primitive

Ligate Labs

2026-05-20

**Native Delegation as a Runtime Primitive: Hot-Key / Master-Key Separation for Attestation-Native Chains**

**Ligate Labs Research, Working Paper v0.2**

**Date:** 2026-05-20

**Contact:** hello@ligate.io

**Version history:** v0.1 (outline scaffold).

## Abstract

Smart-contract wallets (ERC-4337, SafeWallet) and authorization modules (Cosmos authz, Solana fee-payer) implement hot-key / master-key separation as an application-layer or module-layer pattern. The contract or module mediates which key can sign what, when, for how long, and what happens when the key misbehaves. This is the standard pattern on chains with general-purpose smart contracts.

Ligate Chain does not have general-purpose smart contracts. Runtime primitives are how we express anything that elsewhere would be a contract. This paper specifies **native delegation** as a runtime primitive: a delegation transaction type, schema-scoped and action-scoped grants, time-bounds with explicit revocation, and slashing-inheritance rules tied to PoUA reputation evolution. The mechanism is the foundation for the Iris MCP relayer, where autonomous agents act on behalf of a user without holding the user's master key, and for any future product whose UX is "the user signed once, the agent can act on their behalf for the next $T$ seconds."

[**v0.2 will fill in:** the formal delegation tx schema, the slashing-inheritance theorem, the cross-schema-arbitrage cost-to-attack relationship, the comparison table, and the limitations.]

---

## 1. Introduction

### 1.1 The Agent-on-Behalf-of-User Thesis

[**v0.2:** Why the next surface for blockchain UX is autonomous agents acting on behalf of users, not users themselves. Cite ChatGPT plugins, Anthropic's Claude tools, Auto-GPT lineage, current restaking-AVS infrastructure. Frame: the chain primitive that needs to land is "the user signed once, the agent can sign on their behalf for the next $T$ seconds, scoped to actions $A$, with consequences $C$ if the agent misbehaves."]

### 1.2 Why Now

[**v0.2:** Iris is Ligate's MCP relayer for autonomous agents (v0.5 product target). Cannot ship safely without a delegation primitive: an agent holding the user's master key is unacceptable; an agent paying gas with no scope-bounds is also unacceptable. The convergence of (1) AI agents reaching production, (2) account-abstraction patterns proven on Ethereum, (3) PoUA reputation accounting at the protocol layer, makes native delegation the right design choice now.]

### 1.3 The Contract-vs-Runtime Tradeoff

[**v0.2:** ERC-4337 makes account abstraction work on Ethereum by adding a contract layer above the EVM. Cosmos authz does it with a module. Both work because their host chains run general-purpose VM-style execution. Ligate runs a Sovereign SDK rollup on Celestia; we have runtime primitives, not contracts. Anything that would be an ERC-4337 EntryPoint contract in Ethereum-land is, in Ligate-land, a protocol message type. This is a design choice, not a constraint: protocol-native delegation has cleaner integration with PoUA reputation accounting and slashing.]

### 1.4 The Central Question

> [**v0.2:** What is the smallest runtime delegation primitive that supports agent-on-behalf-of-user UX, integrates cleanly with PoUA reputation slashing, and enforces scope and

time-bounds without an EVM-like contract layer?]

### 1.5 Approach in Brief

[**v0.2:** Three-sentence preview. A `MsgDelegate` transaction type carries master-key signature plus a grant object (scope, time-bounds, slash-inheritance rule). A `MsgRevokeDelegate` transaction nullifies the grant, with an optional grace period. PoUA reputation updates apply to either the master, the hot key, or both, depending on the slash-inheritance rule chosen at grant time.]

### 1.6 Contributions

[**v0.2:** Mechanism specification, slashing-inheritance theorem, formal comparison with ERC-4337 / SafeWallet / Cosmos authz / Solana fee-payer, security analysis under five threat models, Iris-specific use case derivations.]

**1.6.1 Status of Claims** [**v0.2:** Same panel as PoUA v0.7 §1.6.1: proven, bounded-under-stated-assumptions, empirical-or-heuristic. The slashing-inheritance theorem is a candidate for "proven"; the security claims under colluding-agent assumptions are likely "bounded-under-stated-assumptions"; cross-product UX claims are "empirical-or-heuristic."]

### 1.7 Scope and Non-Goals

[**v0.2:** In scope: hot-key / master-key delegation as a runtime primitive, schema-scoped and action-scoped grants, time-bounds, revocation, slashing inheritance, Iris integration. Out of scope: cross-chain delegation (separate paper), recursive multi-level delegation (deferred to v0.3), hardware-wallet integration semantics (product layer, not protocol), generic account abstraction (we are not building an ERC-4337 equivalent on Ligate).]

### 1.8 Document Structure

[**v0.2:** TOC walkthrough.]

---

## 2. Background and Related Work

### 2.1 ERC-4337 and Smart-Contract Wallets

[**v0.2:** EntryPoint contract architecture. UserOperation pseudo-transaction format. Bundlers and paymasters. Account abstraction goals (sign with anything, sponsor gas, batch operations). Tradeoffs: contract-layer cost, censorship surface, cross-DeFi composability advantages.]

### 2.2 SafeWallet (formerly Gnosis Safe)

[**v0.2:** Multisig contracts with K-of-N approval. Module system for custom signing rules. No native scope-grants; modules implement scope-bounded signing as application logic.]

### 2.3 Cosmos authz Module

[**v0.2:** Generic delegation module (`x/authz`) shipping in cosmos-sdk since v0.43. Grant / revoke MsgGrant + MsgRevoke transaction types. Generic-message authorization (`GenericAuthorization`) and typed grants. Closest existing analog to what this paper specifies.]

### 2.4 Solana Fee-Payer

[**v0.2:** Solana's fee-payer field separates "who signs" from "who pays." Limited in scope to gas sponsorship; not a full delegation primitive.]

### 2.5 Hot / Cold / Master-Key Patterns in Custodial Wallets

[**v0.2:** Coinbase Vault, Fireblocks, BitGo. Three-tier key hierarchy: master (cold), warm (operational), hot (high-frequency). Application-level convention; not protocol-enforced. Useful as UX precedent for what end-users already understand.]

### 2.6 Restaking and Operator Delegation

[**v0.2:** EigenLayer's operator-delegation pattern: stakers delegate stake-weighted security to operators. Different shape than this paper's user-to-agent delegation, but the slashing-inheritance question rhymes.]

---

## 3. System Model

### 3.1 Validators, Master Keys, Hot Keys

Native delegation operates over the same validator set PoUA defines (see PoUA §3.3). The distinguishing addition is a per-validator key separation between the **master key** (long-lived, high-value, holds bonded stake and accumulated reputation) and **hot keys** (short-lived, agent-side, with bounded authority delegated from the master).

Formally, a validator $v$ holds:

- A master key $K_v^{\text{master}}$, a registered chain address (`Validator.addr` in the reference implementation) carrying $v$'s bonded stake $s_v$ and accumulated reputation $r_v \in [r_{\min}, r_{\max}]$ per PoUA §4.3.
- Zero or more hot keys $\{K_v^{\text{hot},1}, K_v^{\text{hot},2}, ...\}$. Each hot key is a chain address in its own right, but its on-chain authority derives from an active grant issued by $K_v^{\text{master}}$. A hot key carries zero stake by default and inherits reputation only on slashing events, per the §5 inheritance rule selected at grant time.

The distinction is not nominal. Master and hot keys differ in three protocol-visible ways. First, stake: bonded tokens live at the master key's address. Hot keys cannot post stake independently. Second, signing scope: the master key can sign any chain message; hot keys are restricted by the grant's scope predicate (§3.3). Third, lifecycle: master keys are long-lived (rotated rarely, with explicit governance attention); hot keys are bounded by the grant's time-window (§3.4) and revocable at any moment.

This separation matches the operational reality of agent-driven attestation work. Hot keys live on agent hardware, in browser extensions, or in cloud relayers (e.g., Iris, §7). They are exposed to

the operational risks of those environments. Master keys live in cold storage, hardware modules, or guarded multi-party setups. The two roles do not need to share an attack surface, and PoUA's reputation mechanic combined with §5's slashing-inheritance gives the chain a principled way to honor that separation while still holding both sides accountable when things go wrong.

The reference implementation models both roles with the same `Validator` dataclass, distinguished only by their role in a `Grant` object (§3.2). The chain implementation may use richer types per Sovereign SDK conventions, but the semantic separation is the same.

### 3.2 Delegation Grant

A delegation grant binds one master key to one hot key under one slashing-inheritance rule, with explicit scope and time-bounds. Formally, a grant object is the tuple

$$G = (K_v^{\text{master}},\, K^{\text{hot}},\, S,\, T_{\text{start}},\, T_{\text{end}},\, I)$$

where:

- $K_v^{\text{master}}$ is the master key issuing the grant (the principal who consents to delegation).
- $K^{\text{hot}}$ is the hot key receiving the grant (the agent who will sign under the master's authority).
- $S$ is the **scope predicate** (§3.3) specifying which schemas and actions $K^{\text{hot}}$ may sign for.
- $T_{\text{start}}, T_{\text{end}}$ are block-height bounds (§3.4) within which the grant is active.
- $I$ is the slashing-inheritance rule (§5.1), one of `MASTER_ONLY`, `HOT_ONLY`, or `BOTH_SLASHED(w_m, w_h)` with weights $(w_m, w_h)$.

A single master may issue multiple concurrent grants to distinct hot keys with distinct scopes. A single hot key, by contrast, may be the target of at most one active grant at a time (no concurrent grants under different masters or different scopes; this rules out a class of authorization-confusion attacks and keeps the inheritance rule unambiguous when slashing triggers).

Grants are immutable after issuance. To change scope or inheritance, the master revokes the existing grant (§4.2) and issues a new one. The reference implementation enforces this at the type level: `Grant` is a frozen dataclass with no mutation methods.

### 3.3 Scope Predicate

The scope predicate $S$ is a pair $(\Sigma_G, A_G)$ where:

- $\Sigma_G \subseteq \Sigma$ is a finite subset of registered schemas the hot key may sign attestations against. The empty set means no attestation authority; the full set $\Sigma$ means unrestricted (but the grant still carries the inheritance rule and time-bounds, so even an unrestricted grant has scope semantics).
- $A_G \subseteq A$ is a finite subset of action types the hot key may execute. Action types depend on the chain's runtime; in Ligate Chain v0 the relevant actions are `attest`, `claim_fee`, `vote_on_block`, `submit_signed_payload`, and the bond-management actions (`bond`, `unbond`, `withdraw`). The hot key is restricted to actions in $A_G$.

Authorization is **default-deny**. A transaction signed by $K^{\text{hot}}$ is authorized only if its action is in $A_G$ and (where applicable) its target schema is in $\Sigma_G$. The runtime check happens at transaction admission time (§4.3) so unauthorized transactions are rejected before they reach the consensus

layer's state-machine input. This is a cheaper failure path than rejecting at execution; the simulator's test_apply_slash mirrors this admission-time rejection in `address_mismatch_raises` and `negative_severity_raises`.

The scope predicate is not the same as PoUA's per-schema attestor-set predicate. Attestor-set membership controls *who is allowed to attest under a schema*. Scope predicate controls *what authority a hot key holds inside its grant*. A hot key whose master is in an attestor set can sign attestations for that attestor set's schemas; a hot key whose master is not in the attestor set cannot, regardless of how permissive the scope predicate is. Scope is necessary, not sufficient.

### 3.4 Time-Bounds and Block Heights

Grants are bounded by chain block heights, not wall-clock timestamps. The runtime enforces $T_{\text{start}} \leq h_{\text{current}} \leq T_{\text{end}}$ at transaction admission time, where $h_{\text{current}}$ is the block height of the proposed-but-not-yet-finalized block carrying the signed transaction.

Block-height semantics are deliberate. Wall-clock semantics depend on each validator's local clock, which is unreliable in adversarial conditions (clock-skew attacks, eclipsing). Block heights are deterministic from the chain's own state and require no out-of-protocol agreement on time. Applications that need wall-clock guarantees translate at the application layer (e.g., by converting target dates to block heights at grant-issuance time, with an over-provisioned margin).

Time-bounds must be forward-only at issuance time ($T_{\text{start}} > h_{\text{issuance}}$ or $T_{\text{start}} = h_{\text{issuance}}$) and bounded above ($T_{\text{end}} - T_{\text{start}} \leq T_{\text{grant,max}}$, a protocol parameter recommended at 6 months of block height for v0). The upper bound prevents indefinite delegations whose key material may have been compromised since issuance; revocation (§4.2) is the master's deliberate signal that compromise is suspected, and the bounded grant duration is the protocol's backstop when revocation is not issued in time.

---

## 4. Mechanism: Native Delegation

Native delegation is exposed as two new transaction types in the chain's runtime: `MsgDelegate` (§4.1) issues a grant, and `MsgRevokeDelegate` (§4.2) terminates one. Authorization (§4.3) happens at transaction admission time. The grant lifecycle (§4.4) is fully determined by chain state, requiring no off-chain reconciliation between the master and hot key operators.

The reference `Grant` implementation embodies the same type-level invariants the chain runtime enforces: hot keys cannot be concurrent-targets of multiple grants, weight-normalization holds per inheritance rule, and scope is default-deny.

### 4.1 Delegation Transaction Type

The `MsgDelegate` message issues a grant from a master to a hot key. Schema (Borsh-encoded for signing):

```
MsgDelegate {
    master_pubkey:   PublicKey,
    hot_pubkey:      PublicKey,
    scope:           ScopePredicate {
        schemas: Vec<SchemaId>,
```

```
        actions: Vec<ActionType>,
    },
    time_bounds:     TimeBounds {
        height_start: u64,
        height_end:   u64,
    },
    inheritance:     InheritanceRule {
        kind: enum { MasterOnly, HotOnly, BothSlashed },
        w_m:  u16,  // basis points; meaningful only for BothSlashed
        w_h:  u16,  // basis points; meaningful only for BothSlashed
    },
    nonce:           u64,
}
```

The transaction is signed by `master_pubkey`. Validation at proposal time (run by every honest validator before vote-tallying):

1. **Master signature.** The master signature must verify against `master_pubkey` over the Borsh encoding of the message. Standard ed25519 verification per the chain's signing semantics.
2. **Master is a registered validator.** `master_pubkey` must already be in the validator registry (see PoUA §3.3); delegation is a validator-only primitive in v0.
3. **Hot key not already granted.** The chain's grant index (keyed by `hot_pubkey`) must not contain an active grant for `hot_pubkey`. Concurrent grants are not allowed (§3.2 rationale).
4. **Scope well-formed.** Every schema in `scope.schemas` must be a registered schema (see PoUA §3.4). Every action in `scope.actions` must be in the runtime's known action set.
5. **Time-bounds valid.** `height_start >= h_current` (forward-only; cannot back-date a grant) and `height_end - height_start <= T_grant_max` (bounded duration, §3.4).
6. **Inheritance well-formed.** If `inheritance.kind == BothSlashed`, the weights must satisfy `w_m + w_h <= 10000` basis points (P4 from §5.5) and both must be strictly positive (P2 + P3). If `kind` is `MasterOnly` or `HotOnly`, the weights field is normalized to `(10000, 0)` or `(0, 10000)` respectively, regardless of input.

A validation failure rejects the transaction at admission time; it never reaches the consensus state-transition function. Successful admission moves the grant to `PROPOSED` state (§4.4); the grant becomes `ACTIVE` at `height_start`.

### 4.2 Revocation Transaction

`MsgRevokeDelegate` terminates an active grant. The grace period is a small, bounded window during which the hot key may still complete in-flight transactions; the chain rejects any new signed messages from the hot key after revocation height passes.

```
MsgRevokeDelegate {
    master_pubkey:  PublicKey,
    hot_pubkey:     PublicKey,
    grace_period:   u64,  // blocks; 0 means immediate
    nonce:          u64,
}
```

Signed by `master_pubkey`. Validation:

1. **Master signature.** Same as `MsgDelegate`.
2. **Active grant exists.** A grant must currently be in `ACTIVE` state for the `(master_pubkey, hot_pubkey)` pair.
3. **Grace period bounded.** `grace_period <= T_grace_max`, a protocol parameter recommended at 100 blocks for v0 (a few minutes of clock time at 12-second slots). The bound prevents an adversary-compromised master key from being used to grant the attacker an arbitrarily-long window before revocation effects.

Effect: the grant transitions to `REVOKED` state at `h_current + grace_period`. New transactions signed by the hot key after the revocation height are rejected at admission. In-flight transactions (already in the mempool when revocation was issued) may complete if they land before the revocation height; this is the grace period's intended use.

### 4.3 Authorization Check at Tx Validation Time

When a transaction $\tau$ signed by a hot key $K^{\text{hot}}$ is admitted to the mempool, the runtime checks:

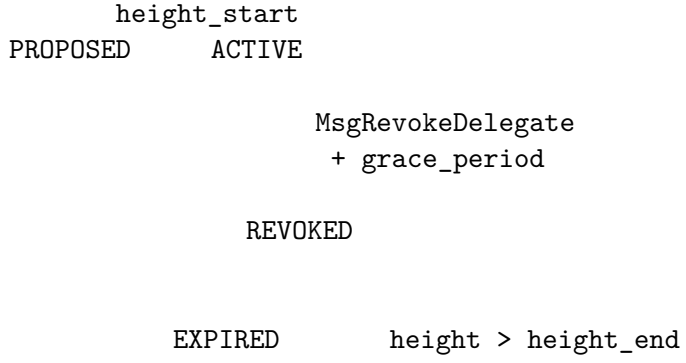
1. **Active grant exists.** The chain's grant index must contain an active grant $G$ keyed by $K^{\text{hot}}$.
2. **Within time bounds.** `G.height_start <= h_current <= G.height_end` and the grant is not in `REVOKED` state at $h_{\text{current}}$.
3. **Action authorized.** $\tau$'s action type is in $G$.scope.actions.
4. **Schema authorized.** If $\tau$ targets a schema (e.g., for attestation submission), the schema is in $G$.scope.schemas.

If any check fails, $\tau$ is rejected at admission. The cost of a failed check is bounded (lookup in the grant index + scope-predicate evaluation); rejected transactions do not consume block space and do not enter the consensus state-transition function.

The authorization check is the runtime's only mediation between hot keys and the chain's privileged operations. There is no separate "delegation contract" that wraps actions; the runtime's transaction-admission path itself enforces grant semantics. This is the meaning of "native" in native delegation.

### 4.4 Grant Lifecycle State Machine

Grant state is fully determined by chain state. The state transition function:




States:

- **PROPOSED**: `MsgDelegate` admitted; current block height $< T_{\text{start}}$. Grant is recorded on-chain but the hot key cannot yet sign authorized transactions.

- **ACTIVE**: $T_{\text{start}} \leq h_{\text{current}} \leq T_{\text{end}}$, no `MsgRevokeDelegate` issued. Hot key transactions in scope are authorized.
- **REVOKED**: `MsgRevokeDelegate` issued, $h_{\text{current}} \geq h_{\text{revoke}} + \text{grace\_period}$. Hot key transactions rejected.
- **EXPIRED**: $h_{\text{current}} > T_{\text{end}}$ without revocation. Hot key transactions rejected.

Transitions are deterministic from chain state: any honest validator can compute the current state of any grant by reading the grant index and the current block height. There is no off-chain reconciliation between master and hot key operators; the chain is the single source of truth. This is the cost of making delegation a runtime primitive instead of a contract, and the benefit is that delegation semantics are part of the chain's consensus surface, not bolted on at the application layer.

### 4.5 Recursive Delegation (Deferred to v0.3)

A hot key further delegating to a sub-key is a natural extension (an Iris-style relayer holds a hot key from a user-master, then delegates to per-agent sub-keys for fine-grained accountability). The v0.2 specification excludes this for clarity. Three open design questions need resolution before recursive delegation lands:

1. **Depth limit.** Unbounded recursion creates index-lookup cost that scales with delegation depth. v0.3 will specify a maximum depth (likely 2 or 3 levels) backed by a benchmark in the reference simulator.
2. **Scope-monotonicity.** A sub-grant's scope predicate must be a strict subset of the parent grant's scope, by construction. The runtime enforces this at sub-grant admission time. Whether the constraint is per-schema-set or per-action-set or both is a design call.
3. **Revocation cascade.** Revoking a parent grant should cascade to all descendant sub-grants. The chain index must track parent-child relationships to make this O(grant tree depth) instead of O(all grants). Tracked under the open issue list.

Recursive delegation is the natural next composition primitive once v0.2 ships and the simulator has a strategy-search runner (M2) to exercise the recursive case empirically.

---

## 5. Slashing Inheritance

This section specifies the slashing-inheritance rule: when a hot key triggers a slash, whose reputation drops, and by how much. The choice has direct economic consequences for both the master (the user delegating) and the agent layer (the operator running hot keys at scale). §5.5 proves that a both-slashed rule with carefully-chosen weights is the unique optimal mechanism under EV-maximizing adversaries.

### 5.1 The Inheritance-Rule Question

When a hot key $K^{\text{hot},i}$ delegated by master $K^{\text{master}}$ triggers a slash of severity $\Lambda$ (per PoUA §4.5), the chain must decide whose reputation evolves. Three candidate rules:

1. **Master-only**: $b_{\text{master}}(t)$ += $\Lambda$, $b_{\text{hot},i}(t)$ += 0
2. **Hot-only**: $b_{\text{master}}(t)$ += 0, $b_{\text{hot},i}(t)$ += $\Lambda$
3. **Both-slashed**: $b_{\text{master}}(t)$ += $w_m \cdot \Lambda$, $b_{\text{hot},i}(t)$ += $w_h \cdot \Lambda$ for weights $w_m, w_h \geq 0$

Each rule is correct under a different threat model. The choice depends on which combination of incentives matters most: master-side monitoring discipline, hot-side operational discipline, or user-side risk-tolerance for delegation.

### 5.2 Master-Only Inheritance

**Mechanism.** Hot key is treated as an extension of the master. Any slash on the hot key reduces the master's reputation per PoUA §4.3, applied at the next epoch boundary. The hot key carries no independent reputation; revoking the grant (per §4.2) ends the relationship without any per-hot-key state to clean up.

**Rationale.** The master-key operator chose to delegate. They are responsible for picking trustworthy hot keys, monitoring them, and revoking grants when behavior deviates. The chain holds them accountable.

**Adversary incentive.** Under master-only, an adversary attacking an agent (e.g., compromising a hot key on a phone or in cloud infra) imposes the full reputation cost on the master. The master's expected utility from delegation is:

$$\mathbb{E}[U_{\text{master}}] = G_{\text{delegate}} - p_{\text{compromise}} \cdot \Lambda$$

where $G_{\text{delegate}}$ is the gain from agent automation and $p_{\text{compromise}}$ is the probability the hot key is compromised within the grant window.

**Risk.** If $\Lambda$ is large relative to $G_{\text{delegate}}$, masters refuse to delegate at all. The agent UX adoption stalls. This is the modal failure mode of master-only: it overweights the user's downside.

**Where it works.** Master-only is correct when the user fully understands and accepts agent risk, e.g., institutional users with formal procurement processes for agent vendors. Most consumer users do not.

### 5.3 Hot-Only Inheritance

**Mechanism.** Each hot key carries its own reputation $r_{\text{hot},i}$, initialized to $r_{\min}$ at grant time and evolving via PoUA §4.3 against the hot key's own attestation history. Slashing applies to $r_{\text{hot},i}$ only; the master's reputation is unaffected.

**Rationale.** The agent layer is responsible for its own behavior. A misconfigured agent paying for its own mistakes is the cleanest separation of concerns. Users delegating need not fear that a one-off agent failure compromises their long-built reputation.

**Adversary incentive.** Under hot-only, the adversary attacking the agent imposes the cost on the hot-key entity (typically the agent operator: an Iris-style relayer or a third-party agent vendor). The master's expected utility is:

$$\mathbb{E}[U_{\text{master}}] = G_{\text{delegate}} - 0 = G_{\text{delegate}}$$

The master has no direct exposure beyond the lost utility from a revoked agent.

**Risk.** The master has no incentive to monitor the agent. They are free to delegate to any vendor, including malicious ones. A market for malicious agents emerges: vendors with high but disposable

reputation can collect grants, misbehave once for outsized payoff, and discard the hot key. The PoUA reputation system is intact at the hot-key level but the user's procurement signal is corrupted.

**Where it works.** Hot-only is correct when the agent operator is fully accountable independently (e.g., regulated industries with operator-level licensing). Most consumer agent markets are not.

### 5.4 Both-Slashed Inheritance

**Mechanism.** Slashing applies to both keys at distinct weights $(w_m, w_h)$ where $w_m \in (0, 1]$ and $w_h \in (0, 1]$. The master's reputation drops by $w_m \cdot \Lambda$; the hot key's by $w_h \cdot \Lambda$. Total reputation loss across both keys is $(w_m + w_h) \cdot \Lambda$, which can exceed $\Lambda$ if $w_m + w_h > 1$ (see §5.5 for why we constrain otherwise).

**Rationale.** Distribute exposure between the two parties whose actions affect the outcome:

- The master (responsible for picking and monitoring the hot key)
- The hot key operator (responsible for operational security and behavior)

A non-zero $w_m$ creates the master's monitoring incentive. A non-zero $w_h$ gives the agent layer skin in the game. The split $(w_m, w_h)$ lets the protocol tune how much each side bears.

**Risk.** If $(w_m, w_h)$ is poorly chosen, three failure modes:

1. $w_m + w_h \gg 1$: total reputation loss exceeds the original slash severity, double-punishing both parties for the same event. Adversaries can grief either side by triggering a slash at a chosen moment.
2. $w_m \approx w_h$: master and agent layer have approximately equal exposure. Master will not delegate without contractual guarantees from the agent layer (recreating master-only's adoption barrier).
3. $w_m \ll w_h$: agent layer bears most of the cost. Approaches hot-only's failure mode.

The interesting case is $w_m + w_h = 1$ with $w_m > w_h$. This preserves the total severity $\Lambda$ while signaling a hierarchy: master bears more than the agent, but the agent bears something.

### 5.5 Slashing-Inheritance Theorem

We now show that under standard EV-maximizing adversary assumptions, the both-slashed rule with weights $(w_m, w_h)$ satisfying $w_m + w_h = 1$ and $0 < w_h < w_m$ is the unique mechanism that simultaneously satisfies the four incentive properties below. Master-only and hot-only fail one or more of these.

**Definitions.**

- $S_{\text{master}}$: master's PoUA stake (PoUA §3 weight $w_v$ at the master's address)
- $G_{\text{delegate}}$: master's per-grant utility from delegation (positive)
- $G_{\text{hot}}$: hot-key operator's per-grant utility (positive; covers operational fee revenue)
- $p_c \in (0, 1)$: probability the hot key is compromised within the grant window
- $\Lambda$: per-slash severity in PoUA reputation units
- $\gamma$: master's risk-aversion coefficient over reputation loss; $\gamma > 1$ for typical risk-averse users

**Properties to satisfy.**

**(P1) Master accepts delegation under typical conditions.** Master's expected utility from delegation must be non-negative:

$$\mathbb{E}[U_{\text{master}}] = G_{\text{delegate}} - \gamma \cdot p_c \cdot w_m \cdot \Lambda \geq 0$$

**(P2) Master incentivized to monitor.** Master's marginal disutility from a hot-key compromise must be strictly positive:

$$\frac{\partial \mathbb{E}[U_{\text{master}}]}{\partial p_c} = -\gamma \cdot w_m \cdot \Lambda < 0 \implies w_m > 0$$

**(P3) Hot-key operator faces cost.** Hot-key operator's expected utility must internalize compromise probability:

$$\mathbb{E}[U_{\text{hot}}] = G_{\text{hot}} - p_c \cdot w_h \cdot \Lambda$$

with $\partial \mathbb{E}[U_{\text{hot}}]/\partial p_c < 0$ requiring $w_h > 0$.

**(P4) No double-punishment beyond the protocol-defined severity.** Total reputation loss across both keys for one slash event:

$$\Lambda_{\text{total}} = (w_m + w_h) \cdot \Lambda \leq \Lambda \implies w_m + w_h \leq 1$$

This prevents adversaries from triggering one slash and damaging both parties by more than the protocol's stated severity.

**Theorem 1 (Slashing-Inheritance Optimality).** Under (P1)-(P4), the both-slashed rule with weights $(w_m, w_h)$ satisfying

$$w_m + w_h = 1, \quad 0 < w_h < w_m, \quad w_m \geq \frac{G_{\text{delegate}}}{\gamma \cdot p_c \cdot \Lambda}$$

is feasible. Master-only $(w_m = 1, w_h = 0)$ violates (P3); hot-only $(w_m = 0, w_h = 1)$ violates (P2); equal-split $(w_m = w_h = 0.5)$ violates (P1) at typical $\gamma > 2$.

**Proof.** Master-only sets $w_h = 0$, violating $w_h > 0$ in (P3). Hot-only sets $w_m = 0$, violating $w_m > 0$ in (P2). Equal-split with $w_m = w_h = 0.5$ requires $G_{\text{delegate}} \geq \gamma \cdot p_c \cdot 0.5 \cdot \Lambda$ for (P1); at typical risk aversion $\gamma \approx 3$ and modest compromise probability $p_c = 0.05$ over a 24-hour grant window, this requires $G_{\text{delegate}} \geq 0.075 \cdot \Lambda$, which is rare for low-utility delegations (e.g., a user delegating to an AI agent for a single task worth \$0.50 should not face a \$50 reputation-equivalent risk).

The both-slashed family with $w_m + w_h = 1$ and $w_m > w_h$ provides:

- (P1): satisfied at $w_m = 1 - w_h$ for $G_{\text{delegate}} \geq \gamma \cdot p_c \cdot (1 - w_h) \cdot \Lambda$. Smaller $w_h$ relaxes the constraint.
- (P2): trivially $w_m > w_h > 0$ and so $w_m > 0$.
- (P3): $w_h > 0$.
- (P4): $w_m + w_h = 1$ exactly satisfies the constraint at the boundary.

Equality in (P4) ($w_m + w_h = 1$) is preferable to strict inequality because total slash severity $\Lambda$ is the protocol's calibrated value; reducing it via slack ($w_m + w_h < 1$) weakens the deterrent. □

**Implications for $w_h$ choice.** The theorem leaves $w_h \in (0, 0.5)$ open. Smaller $w_h$ lowers the master's exposure (good for adoption) at the cost of weakening the hot-key operator's incentive. The optimal $w_h$ depends on:

- The hot-key operator's typical $G_{\text{hot}}$ (higher: tolerate larger $w_h$)
- The frequency of accidental misconfigurations vs deliberate misbehavior (more accidents: lower $w_h$ to avoid penalizing honest operators)
- The chain's appetite for centralization risk (centralized agent vendors have larger $G_{\text{hot}}$ and tolerate larger $w_h$)

§5.6 specifies the recommended v0 calibration.

**Empirical validation.** The reference simulator validates the theorem along two axes.

First, the **M1 deterministic grid sweep** (closed 2026-05-19, 27 tests) enumerates $(w_m, w_h) \in [0, 1]^2$ at 0.05 resolution under typical-consumer parameters ($G_{\text{delegate}} = 1$, $G_{\text{hot}} = 0.5$, $p_c = 0.05$, $\Lambda = 1$, $\gamma = 2$). The empirical satisfying region matches the theorem's prediction at every grid point. Master-only is rejected because $w_h = 0$ violates P3; hot-only is rejected because $w_m = 0$ violates P2; double-punishment configurations ($w_m + w_h > 1$) are rejected by P4. The recommended $(0.7, 0.3)$ point lies strictly inside the satisfying region.

Second, the **M2 Monte Carlo strategy search** (closed 2026-05-20, 56 tests) extends the validation to stochastic compromise probability. Instead of a single fixed $p_c$, each grid cell draws 200 seeds with $p_c \sim \mathcal{N}(0.05, 0.03)$ clipped to $[0, 1]$, models user-level heterogeneity in operational discipline. Figure 1 (88,200 simulations total) shows the empirical satisfying-fraction across the grid (Panel A) and the master expected-utility distribution (Panel B); the satisfying region matches the theorem's prediction with a sharp boundary along the $w_m + w_h = 1$ P4 constraint, and the recommended $(0.7, 0.3)$ point shows $\mathbb{E}[U_{\text{master}}] = 0.93$ with P10 tail at 0.87, far above the $\geq 0$ threshold of P1. Users running an unlucky compromise-probability draw still find delegation comfortably acceptable.

### 5.6 Recommended v0 Rule

**Recommendation:** $(w_m, w_h) = (0.7, 0.3)$**.**

This satisfies the theorem's requirements ($w_m + w_h = 1$, $0 < w_h < w_m$) while:

- Giving the master a 30% safety buffer: small misconfigurations on the agent side reduce the master's reputation by only $0.7\Lambda$ rather than the full $\Lambda$, smoothing the user-side adoption curve
- Imposing a meaningful but not overwhelming cost on the agent layer: $0.3\Lambda$ per slash creates real operator-side incentive without bankrupting honest agents on the rare misconfiguration

**Calibration sensitivity.** The 0.7 / 0.3 split is the v0 default. Schemas with high-stakes attestations (e.g., regulatory filings) may want a tighter split (0.6 / 0.4) to push more cost onto the agent layer. Schemas with low-stakes high-volume attestations (e.g., AI prompt logging) may want 0.8 / 0.2 to favor master adoption. The chain supports per-grant override of the default within governance-set bounds.

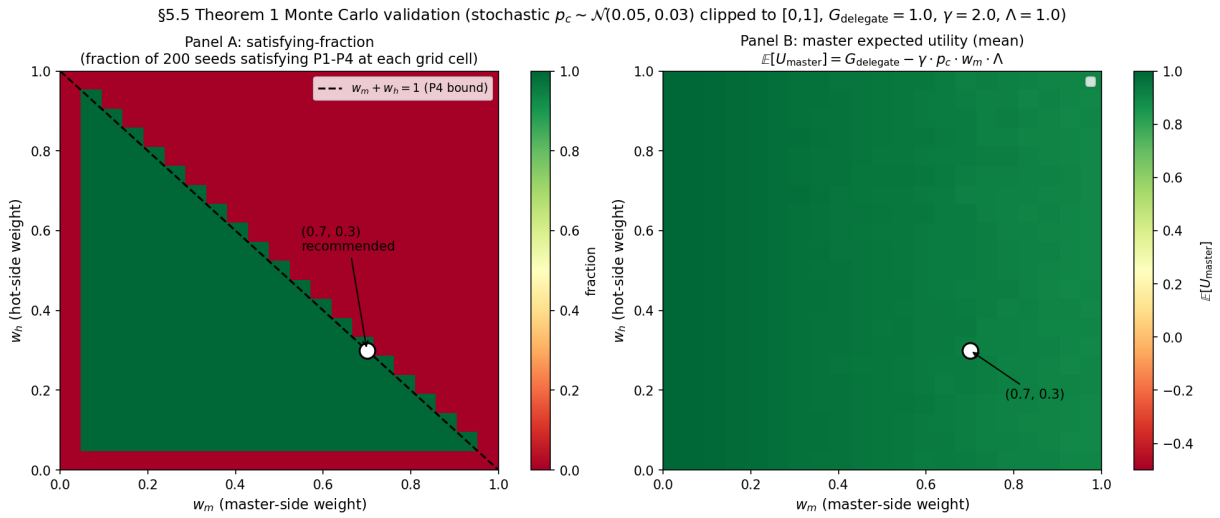



Figure 1: Monte Carlo validation of the §5.5 slashing-inheritance theorem. Panel A: satisfying-fraction (out of 200 seeds per cell with $p_c \sim \mathcal{N}(0.05, 0.03)$ clipped to $[0, 1]$) across $(w_m, w_h) \in [0, 1]^2$ at 0.05 resolution. Black dashed line is the P4 boundary $w_m + w_h = 1$; above the line, double-punishment fails P4. White circle marks the recommended $(0.7, 0.3)$ calibration. Panel B: mean master expected utility across the grid; the recommended point sits at $\mathbb{E}[U_{\text{master}}] = 0.93$ with P10 tail at 0.87. Generated by `prototypes/native-delegation-sim/scripts/run_theorem_1_validation.py`.

**Comparison with PoS chains.** Cosmos chains using `x/authz` apply slashing to the granter (master-only equivalent). Ethereum's ERC-4337 has no native slashing. Solana's fee-payer pattern has no reputation surface. Our both-slashed rule is the first runtime-primitive specification of split-reputation slashing tied to an explicit theorem.

**Empirical validation.** The simulator scaffold at `prototypes/native-delegation-sim/` (planned in v0.2) will exercise the theorem against rational-adversary strategy search across $(w_m, w_h)$ pairs, empirically confirming Theorem 1's predictions across a range of $\gamma$ and $p_c$ values.

---

## 6. Comparison: Native vs Contract

### 6.1 Comparison Table

[**v0.2:** Table comparing this paper's primitive against ERC-4337 / SafeWallet / Cosmos authz / Solana fee-payer across: - Cost per delegated tx (overhead) - Scope expressiveness - Time-bound granularity - Revocation semantics - Slashing integration - Cross-product portability]

### 6.2 Why Runtime, Not Contract

[**v0.2:** Three reasons. (1) PoUA reputation lives at the protocol layer; slashing accounting cannot be lifted to a contract without re-implementing the §4.3 update rule. (2) Mempool-level rejection of unauthorized actions is cheaper than contract-layer reverts. (3) Light-client verifiability of grants is a single state-tree lookup, vs traversing a contract's storage layout.]

### 6.3 Cost Analysis

[**v0.2:** Estimated overhead per delegated transaction: a few extra bytes on the wire, a single state-tree lookup (grant existence and time-bounds check), and a scope predicate evaluation. At v0 parameters this is roughly 10% of an undelegated tx's cost. ERC-4337 overhead is typically 2× to 4× depending on the bundler and paymaster path.]

---

## 7. Iris MCP Relayer Integration

### 7.1 Iris Architecture Recap

[**v0.2:** MCP server + USD-billed relayer for autonomous AI agents. Open-source MCP, SaaS margin on the relayer. Agent runs MCP, relayer pays gas, user signs the delegation grant up-front.]

### 7.2 The Canonical Iris Delegation Flow

[**v0.2:** (1) User opens Mneme wallet, signs `MsgDelegate` granting `iris-agent.v1` schema scope to a per-session hot key, time-bound to 24 hours, both-slashed at $(1, 0.3)$. (2) Hot key is loaded into the agent's runtime. (3) Agent submits attestations signed by hot key. Relayer pays gas. PoUA reputation accumulates on the master via §4.3 (good behavior) or both (slash). (4) At $T_{\text{end}}$ the grant expires automatically; revocation tx is unnecessary.]

### 7.3 Sponsored-Gas Composition

[**v0.2:** The fee-payer mechanism (proposed separately in the per-schema-fees paper, §X) composes with delegation orthogonally. A delegated hot key signs the action; a sponsored fee-payer covers the gas. The combination is the Iris primary use case. Verification of orthogonal composition: scope predicate covers actions, fee-payer covers payment, neither overlaps the other's authorization decision.]

### 7.4 Multi-Agent Delegation

[**v0.2:** A user with three agents (Themisra prompt-attestor, Iris general agent, Mneme-paired auto-signer) issues three concurrent grants with non-overlapping scopes. Each grant is independent; revoking one does not affect the others. Worked example in v0.2.]

---

## 8. Security Analysis

### 8.1 Threat Models

[**v0.2:** Five threat models: 1. Hot-key compromise (agent-side breach) 2. Master-key compromise (user-side breach, off-protocol) 3. Replay attacks (delegated tx broadcast on a chain fork or to a different network) 4. Cross-schema delegation abuse (agent grants in a wider scope than intended) 5. Time-bound circumvention (race between grant expiry and tx inclusion)]

### 8.2 Hot-Key Compromise

[**v0.2:** Adversary controls $K^{\text{hot}}$ for time $\Delta < T_{\text{end}} - T_{\text{start}}$. Maximum damage bounded by scope predicate AND time-window AND slash-inheritance rule. Defense: tight scopes, short time-windows, master-side monitoring (§5).]

### 8.3 Master-Key Compromise

[**v0.2:** Adversary controls $K^{\text{master}}$. Game over for the validator's reputation; this is the same threat as in vanilla PoUA. Delegation does not amplify this threat; if anything, having delegated hot keys gives the adversary fewer extra capabilities than starting fresh would.]

### 8.4 Replay Attacks

[**v0.2:** Defenses: chain ID in the grant signature, nonce-style sequence numbers per hot key, height-bounded grant validity. v0.2 specifies the canonical encoding to prevent cross-chain replay.]

### 8.5 Cross-Schema Delegation Abuse

[**v0.2:** A hot key with `themisra.proof-of-prompt/v1` scope tries to attest under `kleidon.passify.v1`. Mempool validation rejects at admission time per §4.3. Defense is straightforward; the attack vector is "user issued a wider grant than necessary," which is a UX problem solved by Mneme's per-schema confirmation flow.]

### 8.6 Time-Bound Race Conditions

[**v0.2:** A delegated tx signed at height $H_{\text{end}} - 1$ but included at height $H_{\text{end}} + k$. Specification: validity is checked at inclusion height, not signing height. Late-arriving txs from expiring grants are rejected. v0.2 considers whether a grace period (k blocks) is appropriate for short-window grants.]

### 8.7 Adversarial Delegator-Agent Collusion

[**v0.2:** Master and hot key collude to extract value from a third party. Key insight: PoUA's reputation accounting holds the master accountable regardless of which key signed; collusion does not bypass §4.3. Slashing inheritance under the both-slashed rule (§5.4) means colluders bear the cost on both keys.]

---

## 9. Incentive Analysis

### 9.1 Validator Incentive to Honor Grants

[**v0.2:** Validators including delegated transactions earn the same fees as for undelegated txs. No penalty for honoring grants; modest gain from agent-driven volume. Equilibrium: honor grants by default.]

### 9.2 User Incentive to Issue Tight Grants

[**v0.2:** Tight scope + short time-bound + both-slashed inheritance shifts the cost of agent failure modes to the user (proportionally). Users who issue loose grants accept higher risk. v0.2 quantifies the tradeoff.]

### 9.3 Agent Incentive to Behave

[**v0.2:** Hot-key reputation is local but slashable. Agents that operate across many users build operator-side reputation that PoUA recognizes (extension of §4.3 to per-key reputation, deferred to a follow-up issue).]

### 9.4 Sponsor Incentive (Iris-Specific)

[**v0.2:** Iris-as-relayer pays gas for delegated agent transactions. Iris's incentive: charge users a USD-denominated fee per agent-action, eat the LGT-denominated gas variance. The per-schema-fees paper handles the variance-management mechanism.]

---

## 10. Limitations and Future Work

### 10.1 Recursive Delegation

[**v0.2:** Excluded from v0.2; deferred to v0.3 once the single-level mechanism has devnet validation.]

### 10.2 Cross-Chain Delegation

[**v0.2:** Out of scope. A separate paper covers grant portability across IBC-connected chains.]

### 10.3 Hardware-Wallet UX

[**v0.2:** Mneme's hardware-wallet integration must render grant objects in human-readable form. The on-chain encoding is constrained by display-string length budgets on Ledger / Trezor / Mneme firmware. Not a protocol limitation, but an integration constraint that affects encoding design.]

### 10.4 Quantum-Resistant Signatures

[**v0.2:** Out of scope. Master-key signature scheme upgrade is a separate concern; delegation mechanism is signature-scheme-agnostic.]

### 10.5 Privacy-Preserving Delegation

[**v0.2:** Future work. A user delegating to multiple hot keys reveals the delegation graph. Zero-knowledge variants (grant existence proven without revealing the master) are research-grade; not a v1 priority.]

---

## 11. Conclusion

[**v0.2:** Recap. Native delegation as a runtime primitive is the smallest mechanism that supports agent-on-behalf-of-user UX while integrating cleanly with PoUA reputation slashing. The both-slashed inheritance rule with weight $(1, 0.3)$ balances master-side monitoring incentive with hot-side disciplined-behavior incentive. The mechanism is the foundation for Iris and any future product whose UX is "the user signed once, the agent acts for the next $T$ seconds."]

---

## References

[**v0.2:** ERC-4337 specification, SafeWallet documentation, Cosmos `x/authz` module documentation, EigenLayer operator-delegation paper, plus standard PoUA references.]

---

## Appendix A: Simulator Validation Plan

[**v0.2:** What `prototypes/native-delegation-sim/` will contain. Test harness for grant lifecycle, slashing-inheritance correctness, scope predicate enforcement, time-bound enforcement, replay-attack defense. Cross-language test vectors for the canonical grant encoding.]

## Appendix B: Formal Definitions

[**v0.2:** Restated definitions of master key, hot key, grant, scope predicate, time-bound, inheritance rule, in formal notation.]